

Kinesis and Analytics

Data Streams · Firehose · Flink · Athena · Glue · S3 Data Lake

AWS Tutorial · ShopFlow

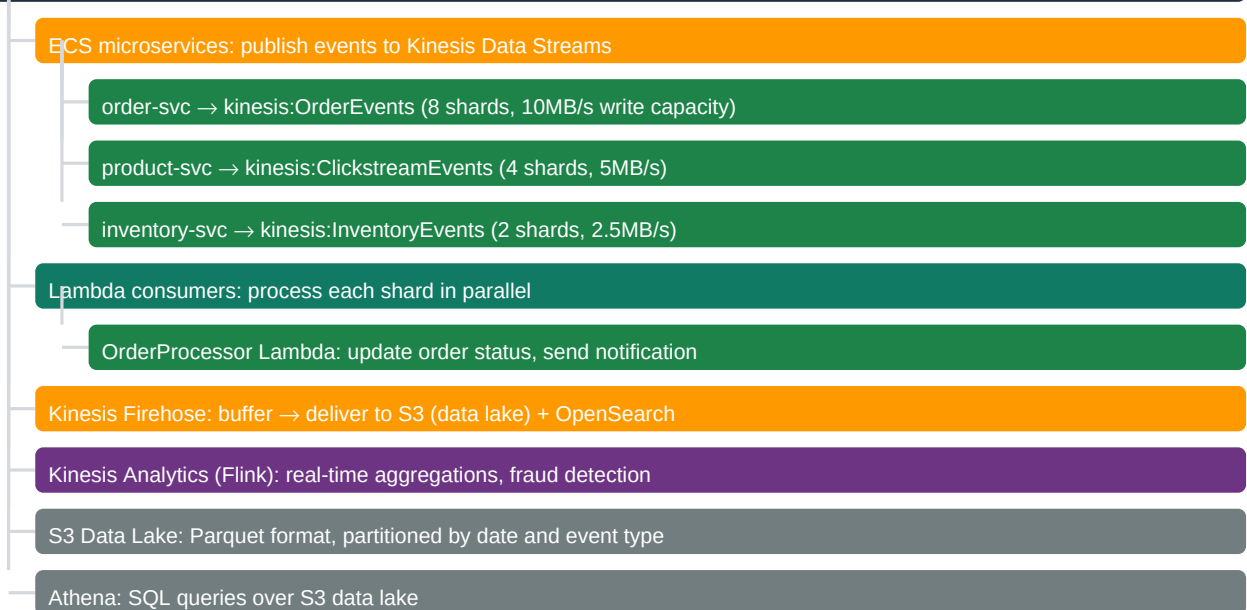
18-00

Chapter Overview — Real-Time Analytics at ShopFlow

ShopFlow uses Kinesis for real-time data streaming: clickstream events, order events, and inventory changes. Kinesis Data Streams captures events; Kinesis Data Firehose delivers to S3 and OpenSearch; Kinesis Data Analytics (Apache Flink) processes streams in real time. Athena queries S3 data lake for batch analytics.

Service	ShopFlow Use Case	Latency	Volume
Kinesis Data Streams	Order events, product views, add-to-cart events	200ms end-to-end	10K events/s normal; 100K Black Friday
Kinesis Data Firehose	Order events → S3 + OpenSearch; ALB logs → S3	5s batch	1 GB/min normal; 10 GB/min Black Friday
Kinesis Data Analytics (Flink)	Real-time: fraud detection, inventory depletion alerts, sending products	10s	10K events/s sustained
Amazon Athena	Daily revenue reports, cohort analysis, product performance (ad-hoc SQL)	Minutes	S3 queries; cost per scan
AWS Glue	ETL: CSV/JSON → Parquet; schema catalog; crawl S3	Hours (batch)	Nightly ETL jobs

ShopFlow Kinesis Data Pipeline



18-01

Kinesis Data Streams CDK — Shard Sizing

Each Kinesis shard supports: 1 MB/s write (1000 records/s), 2 MB/s read. Shard count = $\max(\text{writeRate}/1\text{MB}, \text{readRate}/2\text{MB})$. ShopFlow OrderEvents: 5 MB/s write (10K events × 500 bytes avg) → 5 shards + 3 buffer = 8 shards. Over-provisioning shards is cheap (\$0.015/shard-hour); under-provisioning causes ProvisionedThroughputExceededException.

Stream	Write Rate	Events/s	Shard Count (formula)	Provisioned
OrderEvents	5 MB/s (5000 KB/s)	10,000 × 500 bytes	5 MB/1 MB = 5 shards + 3 buffer	8 shards

Stream	Write Rate	Events/s	Shard Count (formula)	Provisioned
ClickstreamEvents	2 MB/s	20,000 × 100 bytes	2 MB/1 MB = 2 shards + 2 buffer	4 shards
InventoryEvents	0.5 MB/s	5,000 × 100 bytes	1 shard + 1 buffer	2 shards
Black Friday (OrderEvents)	50 MB/s (10x)	100,000 × 500 bytes	50 shards needed	Pre-scale to 50 shards

Kinesis Data Stream CDK

```
const orderStream = new kinesis.Stream(this, "OrderEvents", {
  streamName: "shopflow-order-events",
  shardCount: 8,
  retentionPeriod: Duration.hours(24), // events available for 24 hours
  encryption: kinesis.StreamEncryption.KMS,
  encryptionKey: streamKey,
  streamMode: kinesis.StreamMode.PROVISIONED, // explicit shard count
  // Or: ON_DEMAND: scales automatically, no shard management
  // but 2x more expensive per shard-hour
});
```

18-02

Kinesis Producer — Publishing Events from ECS

ShopFlow ECS services publish events to Kinesis using the Kinesis Producer Library (KPL) for high-throughput batching, or the AWS SDK for simpler single-record puts. KPL aggregates up to 500 records into one Kinesis shard write (reducing cost and increasing throughput). order-svc uses SDK PutRecord for low-latency single-event publishing.

Kinesis Event Publishing — Spring Boot

```
@Service
public class OrderEventPublisher {
  private final KinesisAsyncClient kinesisClient;
  private static final String STREAM = "shopflow-order-events";
  public CompletableFuture<Void> publishOrderCreated(Order order) {
    OrderEvent event = new OrderEvent(
      "ORDER_CREATED", order.getId(), order.getUserId(),
      order.getTotalAmount(), Instant.now());
    PutRecordRequest req = PutRecordRequest.builder()
      .streamName(STREAM)
      .partitionKey(order.getUserId()) // same user = same shard = ordered events
      .data(SdkBytes.fromUtf8String(json.writeValueAsString(event)))
      .build();
    return kinesisClient.putRecord(req)
      .thenApply(resp -> {
        log.debug("Published to shard {} seq {}", resp.shardId(), resp.sequenceNumber());
        return null;
      })
      .exceptionally(ex -> {
        log.error("Kinesis publish failed for order {}", order.getId(), ex);
        throw new RuntimeException("Event publish failed", ex);
      });
  }
}
```

■ Use userId as the partition key for order events — all events for the same user go to the same shard, ensuring in-order processing per user. Without a consistent partition key, ORDER_CREATED and ORDER_SHIPPED for the same order could be in different shards and processed out of order. For global ordering across all events (rare), use a constant partition key (e.g., "all") — but this puts all events on one shard (1 MB/s limit).

18-03

Lambda Kinesis Consumer with Batch Processing

Lambda processes Kinesis shards via an Event Source Mapping (ESM). Each shard gets one concurrent Lambda invocation (8 shards = 8 concurrent Lambdas). Parallelization factor (up to 10) allows multiple Lambdas per shard for higher throughput. Report batch item failures to prevent full batch retry on partial failure.

Lambda Kinesis Consumer CDK

```
const orderProcessorFn = new lambda.Function(this,"OrderProcessor",{...});
// Event Source Mapping: Kinesis → Lambda
orderProcessorFn.addEventSource(new lambdaEventSources.KinesisEventSource(orderStream,{
  batchSize: 100, // process up to 100 records per invocation
  maxBatchingWindow: Duration.seconds(5), // wait 5s to accumulate batch
  parallelizationFactor: 2, // 2 Lambdas per shard = 16 concurrent
  bisectBatchOnError: true, // on failure: split batch in half; retry each half
  startingPosition: lambda.StartingPosition.LATEST,
  reportBatchItemFailures: true, // partial batch success
  retryAttempts: 3, // retry failed records 3 times before DLQ
  destinations: {
    onFailure: new lambdaDestinations.SqsDestination(orderDlq),
  },
}));
```

ESM Parameter	ShopFlow Value	Effect	Trade-off
batchSize	100 records	Process 100 orders per Lambda invocation	Smaller batch = fewer invocations, higher latency
maxBatchingWindow	5 seconds	Wait up to 5s before invoking Lambda	Higher latency, better batching efficiency; 5s acc
parallelizationFactor	2	2 Lambdas process each shard concurrently	Higher throughput per shard; order within shard
bisectBatchOnError	true	Failed batch split in half and retried	Isolates poison pill records instead of retrying en

18-04

Kinesis Data Firehose — S3 Data Lake Delivery

Kinesis Firehose delivers streams to S3, OpenSearch, Redshift, or Splunk. ShopFlow uses Firehose to land all events in S3 as Parquet (columnar format, 10x smaller than JSON) partitioned by year/month/day/hour for Athena query efficiency. Firehose calls a Lambda for JSON → Parquet conversion before S3 delivery.

Firehose CDK — OrderEvents to S3 Parquet

```
const orderFirehose = new firehose.CfnDeliveryStream(this,"OrderFirehose",{
  deliveryStreamName: "shopflow-orders-s3",
  deliveryStreamType: "KinesisStreamAsSource",
  kinesisStreamSourceConfiguration: {
    kinesisStreamArn: orderStream.streamArn,
    roleArn: firehoseRole.roleArn,
  },
  extendedS3DestinationConfiguration: {
    bucketArn: dataLakeBucket.bucketArn,
    prefix: "orders/year={!timestamp:yyyy}/month={!timestamp:MM}/day={!timestamp:dd}/",
    errorOutputPrefix: "errors/orders/",
    bufferingHints: {intervalInSeconds: 60, sizeInMBs: 64},
    compressionFormat: "UNCOMPRESSED", // Lambda converts to Parquet (handles compression)
    dataFormatConversionConfiguration: {
      enabled: true,
      inputFormatConfiguration: {deserializer: {openXJsonSerDe: {}}},
      outputFormatConfiguration: {serializer: {parquetSerDe: {compression: "SNAPPY"}}},
      schemaConfiguration: {
        databaseName: glueCatalog.databaseName,
        tableName: glueOrdersTable.tableName,
        roleArn: firehoseRole.roleArn,
      },
    },
    roleArn: firehoseRole.roleArn,
```

```
},  
});
```

18-05

Athena Queries on S3 Data Lake

Amazon Athena runs SQL queries directly on S3 data (no ETL to a database). Athena uses the AWS Glue Data Catalog for schema. Parquet format + partition pruning (year/month/day) reduce Athena scan cost. ShopFlow uses Athena for: daily revenue reports, product performance analysis, customer cohort analysis, and ad-hoc business analytics.

Athena Queries — ShopFlow Analytics

```
-- Query 1: Daily revenue by product category (last 7 days)  
SELECT date_trunc("day", event_time) as day,  
product_category,  
SUM(order_amount) as total_revenue,  
COUNT(DISTINCT user_id) as unique_buyers  
FROM orders_parquet  
WHERE year = 2024 AND month = 11 AND day BETWEEN 22 AND 29  
GROUP BY 1, 2  
ORDER BY 1 DESC, 3 DESC  
  
-- Query 2: Top 10 products by Black Friday revenue  
SELECT product_id, product_name,  
SUM(quantity) as units_sold,  
SUM(order_amount) as revenue  
FROM orders_parquet  
WHERE year = 2024 AND month = 11 AND day = 29 -- Black Friday  
GROUP BY 1, 2  
ORDER BY revenue DESC LIMIT 10
```

Athena Cost Optimization	Savings	How	ShopFlow Implementation
Parquet format (vs JSON/CSV)	~10x less scan	Columnar: Athena reads only queried columns	Firehose converts to Parquet with SNAPP
Partition pruning (year/month/day)	~90% less scan	WHERE year=2024 AND month=11: only scans 2024-11-*	S3 prefix: s3://shopflow-athena/2024-11-*/
Result caching	\$0 for repeated queries	Same query within 24h uses cached result	Enabled by default in Athena
Workgroup query limits	Prevents runaway queries	Limit max bytes scanned per query (e.g., 10GB)	Athena Workgroup: bytesScannedCutoffP

18-06

Kinesis Data Analytics — Real-Time Flink

Kinesis Data Analytics (Managed Apache Flink) processes streams in real time. ShopFlow Flink application: (1) detect flash sale inventory depletion (item count < 10 → alert), (2) compute trending products (items with > 100 views in last 5 minutes), (3) fraud detection (order > \$500 from new account in < 30 seconds of signup).

Flink SQL — Trending Products (5-min Tumbling Window)

```
-- Flink SQL: count product views per product in 5-minute tumbling windows  
CREATE TABLE clickstream_source (  
  eventType STRING,  
  productId STRING,  
  userId STRING,  
  eventTime TIMESTAMP(3),  
  WATERMARK FOR eventTime AS eventTime - INTERVAL 5 SECONDS  
) WITH (  
  "connector" = "kinesis",  
  "stream" = "shopflow-clickstream-events",  
  "aws.region" = "us-east-1"  
);  
  
SELECT TUMBLE_END(eventTime, INTERVAL 5 MINUTE) as window_end,  
productId,
```

```
COUNT(*) as view_count
FROM clickstream_source
WHERE eventType = "PRODUCT_VIEW"
GROUP BY TUMBLE(eventTime, INTERVAL 5 MINUTE), productId
HAVING COUNT(*) > 100; -- trending: > 100 views in 5 minutes
```

■ Flink watermarks define how late data can arrive before the window is closed. `WATERMARK FOR eventTime AS eventTime - INTERVAL 5 SECONDS` means: Flink waits up to 5 seconds for late-arriving events before closing the 5-minute window and emitting results. Without watermarks, Flink would close windows based on processing time (when events are processed by Flink), which does not handle network delays and out-of-order events from Kinesis shards.

18-07

Kinesis On-Demand vs Provisioned Mode

Kinesis On-Demand mode automatically scales shard capacity. No shard management, no `ProvisionedThroughputExceededException`. Cost: \$0.08 per million records + \$0.04/GB (vs Provisioned: \$0.015/shard-hour). ShopFlow uses On-Demand for ClickstreamEvents (highly variable traffic) and Provisioned for OrderEvents (predictable, high-volume).

Mode	Cost	Scaling	Use When
Provisioned (ShopFlow OrderEvents)	\$0.015/shard-hour × 8 = \$0.12/hr (\$86/mo)	Manual: call <code>UpdateShardCount</code> API directly	Fixed capacity; high sustained traffic
On-Demand (ShopFlow ClickstreamEvents)	\$0.08/million records + \$0.04/GB	Automatic: scales up to 200MB/s per shard	Traffic is variable (Black Friday spikes)
On-Demand Black Friday cost	At 100K events/s × 1KB × 86400 × 3 days = 25.9TB	Auto-scales: no action needed	\$0.08 × 25,900 = \$2,072 vs Provisioned

18-08

AWS Glue — Data Catalog and ETL

AWS Glue Data Catalog stores table schemas for Athena. Glue Crawlers discover schemas from S3 data automatically. Glue ETL jobs transform raw JSON events to Parquet and enrich data (join order events with product catalog). Nightly Glue job: last 24h of orders enriched with product names and customer tiers.

Glue Crawler CDK

```
const ordersCrawler = new glue.CfnCrawler(this,"OrdersCrawler",{
  name: "shopflow-orders-crawler",
  role: glueRole.roleArn,
  databaseName: shopflowDb.databaseInput.name,
  targets: {
    s3Targets: [{
      path: `s3://${dataLakeBucket.bucketName}/orders/`,
      exclusions: ["errors/**"],
    }],
  },
  schedule: {scheduleExpression: "cron(0 2 * * ? *)"}, // 2am daily
  schemaChangePolicy: {
    updateBehavior: "UPDATE_IN_DATABASE",
    deleteBehavior: "LOG", // log schema drops; do not delete table
  },
});
```

Glue Component	Purpose	ShopFlow Usage
Glue Data Catalog	Central schema registry (databases, tables, partitions)	Athena uses Glue catalog to understand S3 Parquet schemas
Glue Crawlers	Auto-discover and update table schemas from S3 data	Nightly crawler: updates partitions as new daily S3 data arrives
Glue ETL Jobs (Python/Spark)	Transform and enrich data between S3 locations	Nightly: raw JSON → Parquet; join with product/user dimension tables
Glue DataBrew	No-code visual data preparation	Not used (Glue ETL sufficient for ShopFlow)

18-09

S3 Data Lake Architecture

ShopFlow S3 data lake uses a Bronze/Silver/Gold layered architecture. Bronze: raw events (JSON, exactly as received from Kinesis). Silver: cleaned and enriched data (Parquet, validated schema, joined with dimension tables). Gold: aggregated business metrics (daily revenue summaries, product performance, customer lifetime value).

Layer	S3 Path	Format	Data Quality	Retention
Bronze (Raw)	s3://datalake/raw/orders/YYYY/MM/DD/	JSON, gzipped	As-is from Kinesis; may have null fields, schema not validated	1 year (compliance)
Silver (Cleaned)	s3://datalake/silver/orders/YYYY/MM/DD/	Parquet, Snappy compressed	Validated schema; nulls handled; enriched with product names	8 years
Gold (Aggregated)	s3://datalake/gold/daily-revenue/YYYY/MM/	Parquet, small files	Aggregated business metrics; ready for BI dashboards	3 years (financial records)

S3 Lifecycle Policy CDK

```
dataLakeBucket.addLifecycleRule({
  prefix: "raw/",
  transitions: [{
    storageClass: s3.StorageClass.INTELLIGENT_TIERING, // auto-tier by access freq
    transitionAfter: Duration.days(30), // after 30 days
  }, {
    storageClass: s3.StorageClass.GLACIER,
    transitionAfter: Duration.days(90),
  }],
  expiration: Duration.days(365), // Bronze: 1-year retention
});
```

18-10

Kinesis Fan-Out — Enhanced vs Standard Consumers

Standard Kinesis consumers share the 2MB/s read limit per shard — with 3 consumers, each gets 0.67MB/s. Enhanced Fan-Out (EFO) gives each consumer a dedicated 2MB/s read throughput. ShopFlow uses EFO for Firehose (S3 delivery) and Flink (analytics) to prevent them from starving each other at high throughput.

Consumer Type	Throughput per Shard	Latency	Cost	ShopFlow Usage
Standard (GetRecords polling)	Shared 2MB/s across all standard consumers	250-2000ms (polling interval)	No extra charge	Lambda ESM (OrderProcessing)
Enhanced Fan-Out (EFO) (HTTP/2 push)	Dedicated 2MB/s per consumer	~70ms (push vs polling)	\$0.015/shard-hour per EFO consumer	Firehose + Flink — each gets 2MB/s

Register EFO Consumer CDK

```
// Register Firehose as EFO consumer for dedicated 2MB/s per shard
const firehoseConsumer = new kinesis.CfnStreamConsumer(this, "FirehoseEfo", {
  streamArn: orderStream.streamArn,
  consumerName: "shopflow-firehose-consumer",
});
// Register Flink as EFO consumer
const flinkConsumer = new kinesis.CfnStreamConsumer(this, "FlinkEfo", {
  streamArn: orderStream.streamArn,
  consumerName: "shopflow-flink-consumer",
});
// Lambda uses standard polling (not EFO) – Lambda ESM handles automatically
// EFO cost: $0.015/shard-hour per EFO consumer
// 8 shards × 2 EFO consumers × $0.015 × 720h/mo = $172.80/month
```

18-11

Kinesis Scaling — Black Friday Pre-Scaling Playbook

Black Friday: ShopFlow expects 10x normal Kinesis volume. OrderEvents: 8 shards normal to 50 shards Black Friday (100K events/s x 500 bytes = 50MB/s write). Shard scaling is a two-step process: first split shards (2x), then split again (2x). Each step takes 5-10 minutes. Pre-scale 48 hours before Black Friday; scale back down 48 hours after.

Phase	Timing	Action	Shard Count	Why
Normal Operation	All year	No change	8 shards (OrderEvents)	50K events/s x 500 bytes = 5MB/s
Pre-Scale Step 1	48h before Black Friday	UpdateShardCount to 20 shards	20 shards	Split each shard; wait 10 min for completion
Pre-Scale Step 2	47h before Black Friday	UpdateShardCount to 50 shards	50 shards	Now at 50MB/s capacity; ready for Black Friday
Black Friday Active	Day-of	Monitor IteratorAge	50 shards	Alert if IteratorAge > 1 min (0.5MB/s)
Post-Event Scale Down	48h after	UpdateShardCount: 50 to 20 to 8 (two steps)	8 shards	Cost: 50 shards x \$0.015/hr vs 8 shards x \$0.015/hr

Kinesis UpdateShardCount CLI

```
# Pre-scale OrderEvents: 8 shards to 20 shards (step 1)
aws kinesis update-shard-count
--stream-name shopflow-order-events
--target-shard-count 20
--scaling-type UNIFORM_SCALING

# Wait for completion (~10 min), then step 2: 20 to 50 shards
aws kinesis describe-stream-summary
--stream-name shopflow-order-events
--query StreamDescriptionSummary.StreamStatus
# Output: "UPDATING" then "ACTIVE"
```

■ Use On-Demand mode (kinesis.StreamMode.ON_DEMAND) to eliminate Black Friday shard management entirely. Tradeoff: On-Demand is ~2x more expensive per shard-hour but removes operational burden of pre-scaling. For ShopFlow OrderEvents at 50MB/s for 3 days: Provisioned = \$162; On-Demand = ~\$250 extra but zero pre-scaling work.

18-12

Dead Letter Queue Pattern for Failed Records

Without a DLQ, a single malformed order event blocks all subsequent events on that shard for 24 hours. Solution: bisect on error (split batch in half on failure), retry 3 times, then send to SQS DLQ with metadata for manual review and replay. reportBatchItemFailures ensures only failed records retry; successful records in the batch are checkpointed normally.

DLQ Processing Architecture

Kinesis OrderEvents Shard

Lambda ESM: batchSize=100, bisectBatchOnError=true, retryAttempts=3

Success: process 100 records, advance checkpoint

Failure: split batch 100 to 50 + 50; retry each half

Still failing after 3 retries: single bad record isolated

reportBatchItemFailures: only failed records sent to DLQ

SQS DLQ: shopflow-order-processor-dlq

Attributes: shard_id, sequence_number, error_message, retry_count

CloudWatch Alarm: DLQ depth > 0 triggers PagerDuty alert

Report Batch Item Failures — Lambda Handler

```
public StreamsEventResponse handleRequest(KinesisEvent event, Context ctx) {
    List<StreamsEventResponse.BatchItemFailure> failures = new ArrayList<>();
    for (KinesisEvent.KinesisEventRecord record : event.getRecords()) {
        try {
```

```

processRecord(record);
} catch (Exception e) {
failures.add(StreamsEventResponse.BatchItemFailure.builder()
.withItemIdentifier(record.getKinesis().getSequenceNumber())
.build());
}
}
return StreamsEventResponse.builder()
.withBatchItemFailures(failures)
.build(); // only failed records retried; others checkpointed
}

```

18-13

Kinesis Monitoring — Key Metrics and Alarms

Critical Kinesis metrics: `GetRecords.IteratorAgeMilliseconds` (consumer lag), `PutRecord.Success` (write errors), `ReadProvisionedThroughputExceeded` (consumer starving), `WriteProvisionedThroughputExceeded` (producer throttled). `IteratorAge > 60,000ms` (1 minute) indicates consumer falling behind — scale shards or Lambda concurrency.

Metric	Alarm Threshold	ShopFlow Action
GetRecords.IteratorAgeMilliseconds	Max > 60,000ms (1 min) for 5 min	Consumer falling behind: increase Lambda concurrency or parallelism
WriteProvisionedThroughputExceeded	Sum > 0 for 5 min	Producers throttled: add shards or switch to On-Demand mode
ReadProvisionedThroughputExceeded	Sum > 0 for 5 min	Standard consumer starved: add EFO or increase polling interval
PutRecord.Success	Success rate < 99% for 5 min	Write failures: check IAM, VPC endpoints, retry in producer
Lambda ConcurrentExecutions	> 80% of account limit	Pre-scale Lambda concurrency reservation before Black Friday

IteratorAge Alarm CDK

```

new cloudwatch.Alarm(this, "KinesisIteratorAge", {
metric: new cloudwatch.Metric({
namespace: "AWS/Kinesis",
metricName: "GetRecords.IteratorAgeMilliseconds",
dimensionsMap: {StreamName: "shopflow-order-events"},
statistic: "Maximum",
period: Duration.minutes(1),
}),
threshold: 60000, // 1 minute consumer lag
evaluationPeriods: 5,
alarmActions: [snsTopic.topicArn],
});

```

18-14

Kinesis VPC Endpoints and Security

ShopFlow ECS services run in private subnets with no internet access. Kinesis traffic must route via VPC Interface Endpoints to avoid internet traversal. IAM policies restrict which services can publish to which streams — `order-svc` can only write to `OrderEvents`; `product-svc` can only write to `ClickstreamEvents`.

Kinesis VPC Endpoint CDK

```

new ec2.InterfaceVpcEndpoint(this, "KinesisEndpoint", {
vpc,
service: ec2.InterfaceVpcEndpointAwsService.KINESIS_STREAMS,
subnets: {subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS},
privateDnsEnabled: true, // resolve kinesis.us-east-1.amazonaws.com to private IP
securityGroups: [kinesisEndpointSg],
});
// IAM: order-svc can only publish to OrderEvents stream
orderStream.grantWrite(orderServiceTaskRole);

```

```
clickstreamStream.grantWrite(productServiceTaskRole);
// Separate read grants for Lambda consumers
orderStream.grantRead(orderProcessorRole);
```

Security Control	Implementation	Protects Against
VPC Interface Endpoint	All Kinesis API calls route through private endpoint	Data exfiltration via internet; network-level interception
KMS Encryption at rest	Stream encrypted with customer-managed KMS key	S3/Kinesis storage access by unauthorized AWS accounts
IAM per-service write grants	order-svc: write OrderEvents only; product-svc: write ClickstreamEvents only	Compromised service cannot write to other streams
Security Group on endpoint	Only ECS tasks SG and Lambda SG allowed to reach Kinesis endpoint	Kinesis endpoint: non-app resources cannot reach Kinesis

18-15 Cost Analysis — Full Kinesis Architecture

Full ShopFlow Kinesis cost (monthly, normal load): Kinesis Streams (\$131/mo), Kinesis Firehose (\$120/mo), Kinesis Analytics Flink (\$438/mo), Athena (\$50/mo), Glue (\$45/mo), S3 data lake (\$23/mo). Total: ~\$807/mo. Black Friday (3 days, 10x volume): +\$1,200 incremental. Kinesis Analytics (Flink) dominates cost at 54% of total.

Service	Normal Monthly Cost	Cost Driver	Optimization
Kinesis Data Streams (Provisioned)	\$131/mo	8+4+2 shards x \$0.015/hr x 720h	On-Demand for Clickstream saves ~\$30/mo;
Kinesis Data Firehose	\$120/mo	300 GB/mo x \$0.029/GB processed	Parquet conversion reduces downstream S3
Kinesis Analytics Flink	\$438/mo	1 KPU x \$0.11/KPU-hr x 720h x ~5 apps	Use Flink SQL instead of DataStream API: re
Amazon Athena	\$50/mo	10 TB scanned/mo x \$5/TB (Parquet, partitioned)	Web: 100 TB JSON x \$5/TB = \$500/mo
AWS Glue Crawlers	\$45/mo	3 crawlers x 1 DPU x 5 min/day x 30 days x \$0.44/DPU-hr	(not daily) for stable schemas s
S3 Data Lake Storage	\$23/mo	1 TB total x \$0.023/GB (Standard)	Intelligent Tiering + Glacier for Bronze saves

18-16 Kinesis vs SQS vs EventBridge — Decision Matrix

ShopFlow uses all three messaging services. Each has distinct characteristics: Kinesis (ordered stream, multiple consumers, replay, analytics), SQS (queue, one consumer per message, retry, visibility timeout), EventBridge (event bus, 100+ targets, schema registry, scheduled rules). Choosing the wrong service causes architectural issues.

Criteria	Kinesis Data Streams	SQS Standard/FIFO	EventBridge
Message ordering	Per shard (same partition key = same shard)	FIFO only (300 TPS limit per queue)	No ordering guarantees
Multiple consumers	Yes (EFO: each consumer gets dedicated shard)	No (2MB/s per shard) consumed once; fan-out requires SQS	Yes (requires targets SQS)
Replay or re-read	Yes (up to 7 days extended retention)	No (consumed messages deleted)	No (events are transient)
Max record size	1 MB per record	256 KB per message	256 KB per event
ShopFlow usage	OrderEvents, ClickstreamEvents (analytics)	Asynchronous queueing; mail-svc, fulfillment	Decoupled microservice events; inventory

18-17 Idempotent Consumer — DynamoDB Deduplication

Kinesis Lambda consumers can receive duplicate records due to: Lambda retry on failure, re-shard operations, parallel processing. OrderProcessor must be idempotent: processing the same ORDER_CREATED event twice must not create two orders or charge the customer twice. Use DynamoDB conditional writes with sequence number as dedup key (TTL = Kinesis retention + 1 day).

Idempotent Order Processor — DynamoDB Dedup

```
public void processRecord(KinesisEvent.KinesisEventRecord record) {
    String seqNum = record.getKinesis().getSequenceNumber();
    OrderEvent event = parseEvent(record);
```

```
// Try to claim this sequence number (idempotency key)
try {
dynamoDB.putItem(PutItemRequest.builder()
.tableName("shopflow-processed-events")
.item(Map.of(
"sequenceNumber", AttributeValue.fromS(seqNum),
"processedAt", AttributeValue.fromS(Instant.now().toString()),
"ttl", AttributeValue.fromN(
String.valueOf(Instant.now().plusSeconds(172800).getEpochSecond()))
))
.conditionExpression("attribute_not_exists(sequenceNumber)")
.build());
} catch (ConditionalCheckFailedException e) {
log.info("Duplicate event seq={}, skipping", seqNum);
return; // already processed; idempotent: do nothing
}
processOrder(event); // only reached if record is new
}
```

■ TTL = 48 hours (172800 seconds): extends beyond Kinesis 24h retention by 1 day. If Lambda replays from TRIM_HORIZON (Kinesis 24h), the DynamoDB dedup table catches any records processed in the last 48h. TTL auto-expires the dedup entry so the table does not grow indefinitely. Use sequence number (not event ID from payload) as the dedup key — sequence numbers are guaranteed unique by Kinesis per shard, even for business-duplicate events (two identical orders from same user).

18-18

Firehose Dynamic Partitioning for Efficient Queries

Firehose Dynamic Partitioning extracts values from records (using JQ) to create per-key S3 prefixes. ShopFlow uses dynamic partitioning to partition order events by region (us-east-1, eu-west-1) and customer tier (premium, standard) enabling Athena to prune partitions per region and tier — scanning 1/6 of data instead of all data.

Dynamic Partitioning CDK

```
extendedS3DestinationConfiguration: {
bucketArn: dataLakeBucket.bucketArn,
prefix: "orders/region={!{partitionKeyFromQuery:region}}/tier={!{partitionKeyFromQuery:tier}}/year=
!{timestamp:yyyy}/month={!{timestamp:MM}}/",
dynamicPartitioningConfiguration: {enabled: true},
processingConfiguration: {
enabled: true,
processors: [{
type: "MetadataExtraction",
parameters: [{
parameterName: "MetadataExtractionQuery",
parameterValue: "{region:.orderRegion,tier:.customerTier}", // JQ
},{
parameterName: "JsonParsingEngine",
parameterValue: "JQ-1.6",
}],
}],
},
bufferingHints: {intervalInSeconds: 60, sizeInMBs: 64},
}
```

■ Dynamic partitioning with 6 partition values (3 regions x 2 tiers) means Firehose creates 6 S3 prefix paths. Each prefix buffers independently to 64MB before writing to S3. Minimum buffer: 64MB per partition. Increasing bufferingHints.sizeInMBs to 128MB reduces small file overhead — Athena scans many small files much slower than fewer large files (file open/close overhead is per-file, not per-byte).

18-19

Glue ETL — JSON to Parquet Nightly Transform

Glue ETL jobs run Spark-based transformation from raw Bronze (JSON) to enriched Silver (Parquet). Nightly Glue job: last 24 hours of order events enriched with product names from DynamoDB and customer tier from RDS, then written to Silver S3 as Parquet with Snappy compression. Glue job runs at 2am daily, completing before business hours.

Glue ETL Job Script (PySpark)

```
import sys
from awsglue.context import GlueContext
from awsglue.transforms import *
from pyspark.context import SparkContext
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
# Read Bronze layer (raw JSON from Firehose)
raw_orders = glueContext.create_dynamic_frame.from_catalog(
    database="shopflow_db",
    table_name="orders_raw",
    push_down_predicate="(year == 2024 and month == 11 and day == 29)",
)
# Apply schema validation and cleaning
cleaned = ApplyMapping.apply(frame=raw_orders,
    mappings=[("orderId", "string", "order_id", "string"),
    ("userId", "string", "user_id", "string"),
    ("totalAmount", "double", "order_amount", "double"),
    ("eventTime", "string", "event_time", "timestamp")]
)
# Write Silver layer (Parquet, partitioned)
glueContext.write_dynamic_frame.from_options(
    frame=cleaned,
    connection_type="s3",
    connection_options={"path": "s3://shopflow-datalake/silver/orders/"},
    format="parquet",
    format_options={"compression": "snappy"},
)
```

18-20

Athena Workgroups and Cost Controls

Athena Workgroups separate query execution environments for different teams (data-engineering, product-analytics, finance). Each workgroup has independent cost controls: bytes-scanned limit per query, data usage alerts, and result location. Finance queries are limited to 10GB per query; product analytics unlimited but monitored.

Workgroup	Team	Bytes Scanned Limit	Result Location	Cost Alert
shopflow-data-eng	Data Engineering	No limit (trusted)	s3://athena-results/data-eng/	Alert > \$500/day
shopflow-product	Product Analytics	50 GB per query	s3://athena-results/product/	Alert > \$200/day
shopflow-finance	Finance	10 GB per query (prevents runaway jobs)	s3://athena-results/finance/	Alert > \$100/day
shopflow-adhoc	Developer sandbox	5 GB per query	s3://athena-results/adhoc/	Alert > \$50/day

Athena Workgroup CDK

```
new athena.CfnWorkGroup(this,"FinanceWorkgroup",{
    name: "shopflow-finance",
    workGroupConfiguration: {
        bytesScannedCutoffPerQuery: 10 * 1024 * 1024 * 1024, // 10 GB limit
        enforceWorkGroupConfiguration: true, // users cannot override
        resultConfiguration: {
            outputLocation: "s3://shopflow-athena-results/finance/",
            encryptionConfiguration: {
                encryptionOption: "SSE_S3",
            },
        },
    },
})
```

```

},
publishCloudWatchMetricsEnabled: true,
},
recursiveDeleteOption: false,
});

```

■ `enforceWorkGroupConfiguration: true` prevents users from running queries outside the workgroup settings. Without this, an analyst can switch to the primary workgroup (no limits) using the Athena console or SDK, bypassing cost controls. For finance queries over sensitive data, also add an Athena resource-based policy restricting which IAM roles can use the workgroup — this adds a second layer: even if someone switches workgroups, IAM denies the query.

18-21

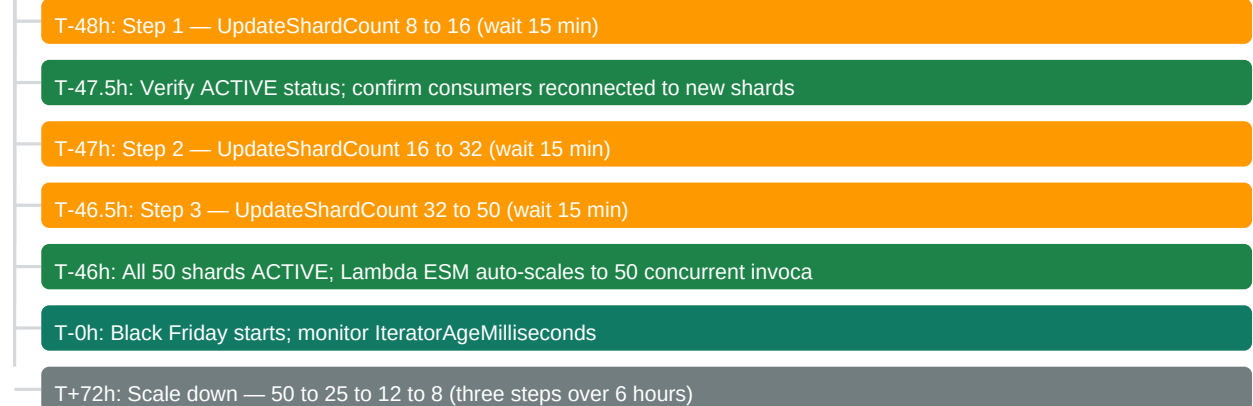
Kinesis Data Streams — Resharding Strategies

Resharding (adding or removing shards) must be done in steps: Kinesis supports at most doubling or halving shard count per `UpdateShardCount` call. Maximum 10 `UpdateShardCount` calls per rolling 24-hour period per stream. To go from 8 to 50 shards: 8 → 16 → 32 → 50 (three steps, each taking 5-10 minutes). Plan resharding schedule well before Black Friday.

Resharding Direction	Kinesis Operation	Maximum Change per Step	Minimum Time Between Steps
Scale Up (add shards)	<code>UpdateShardCount</code> with higher target	Double current shard count (8 to 16, not 8 to 50 directly)	15 minutes (stream must reach ACTIVE state)
Scale Down (remove shards)	<code>UpdateShardCount</code> with lower target	Halve current shard count (16 to 8, not 50 to 8 directly)	15 minutes (parent shard must be fully closed)
On-Demand auto-scale	Automatic (no action required)	Up to 200MB/s per stream (no shard limit visible)	Seconds (transparent to application)

Black Friday Resharding Timeline — ShopFlow

ShopFlow Kinesis Resharding Plan (OrderEvents: 8 to 50 shards)



■ Lambda Event Source Mapping automatically detects new shards after resharding — no code change needed. However, there is a 5-10 minute delay before Lambda discovers new shards and starts polling them. During this window, records on new shards accumulate but are not processed. Monitor `IteratorAgeMilliseconds` during resharding — if it spikes and then drops, Lambda has discovered the new shards. If it continues rising, investigate ESM shard discovery issue.

18-22

Real-Time Fraud Detection with Kinesis Analytics

ShopFlow Fraud Detection Flink application: detect suspicious orders in real-time. Rules: (1) Order > \$500 from account created < 24 hours ago. (2) Three or more failed payment attempts in 10 minutes from same IP. (3) Order shipped to address different from billing address for first-time customer. Alert fires within 5 seconds; order held for manual review.

Fraud Detection Flink SQL

```

-- Rule 1: High-value order from new account
CREATE VIEW suspicious_high_value AS
SELECT o.orderId, o.userId, o.orderAmount,

```

```
o.eventTime, u.accountCreatedAt,
TIMESTAMPDIFF(HOUR, u.accountCreatedAt, o.eventTime) as account_age_hours
FROM order_events o
JOIN user_profiles FOR SYSTEM_TIME AS OF o.eventTime AS u
ON o.userId = u.userId
WHERE o.orderAmount > 500.00
AND TIMESTAMPDIFF(HOUR, u.accountCreatedAt, o.eventTime) < 24;
-- Rule 2: Repeated failed payments from same IP (10-min sliding window)
SELECT ipAddress, COUNT(*) as failed_attempts,
HOP_END(eventTime, INTERVAL 1 MINUTE, INTERVAL 10 MINUTE) as window_end
FROM payment_events
WHERE eventType = "PAYMENT_FAILED"
GROUP BY ipAddress,
HOP(eventTime, INTERVAL 1 MINUTE, INTERVAL 10 MINUTE)
HAVING COUNT(*) >= 3; -- 3+ failed attempts in 10 min sliding window
```

Fraud Rule	Detection Window	Action	False Positive Rate	ShopFlow Tuning
New account high-value order (<math>P < 0.01</math> if 2nd order join)	Real-time (24h window)	Hold order; send to manual review queue	2% (legitimate new customers making large purchases)	Monitor for large purchases
Failed payments from same IP (<math>P < 0.01</math> in 10 min HOP window)	10-min HOP window	Block IP in WAF; alert security team	0.5% (shared office IP, WAF gateway)	Monitor for repeated CIDR ranges
New customer: billing != shipping address (first order check)	Real-time	Add to review queue (not auto-block)	1.5% (gift purchases and shipping address errors)	Alert if shipping address is not in the same city as billing address

18-23

Kinesis Firehose to OpenSearch — Real-Time Indexing

Kinesis Firehose delivers order events directly to Amazon OpenSearch for real-time search indexing. ShopFlow uses this path to keep the order search index current: when a customer searches "my orders", OpenSearch returns results within 60 seconds of order placement. Firehose buffers for 60 seconds (or 5MB), then bulk-indexes to OpenSearch.

Firehose to OpenSearch CDK

```
const orderSearchFirehose = new firehose.CfnDeliveryStream(this, "OrderSearchFirehose", {
  deliveryStreamName: "shopflow-orders-opensearch",
  deliveryStreamType: "KinesisStreamAsSource",
  kinesisStreamSourceConfiguration: {
    kinesisStreamArn: orderStream.streamArn,
    roleArn: firehoseRole.roleArn,
  },
  amazonopensearchserviceDestinationConfiguration: {
    domainArn: openSearchDomain.domainArn,
    indexName: "orders",
    indexRotationPeriod: "OneMonth", // rotate: orders-2024-11, orders-2024-12
    bufferingHints: {intervalInSeconds: 60, sizeInMBs: 5},
    retryOptions: {durationInSeconds: 300}, // retry for 5 min on OpenSearch error
    s3BackupMode: "FailedDocumentsOnly",
    s3Configuration: {
      bucketArn: backupBucket.bucketArn,
      roleArn: firehoseRole.roleArn,
    },
    roleArn: firehoseRole.roleArn,
    vpcConfiguration: {
      subnetIds: privateSubnetIds,
      securityGroupIds: [firehoseToOpenSearchSg.securityGroupId],
      roleArn: firehoseRole.roleArn,
    },
  },
});
```

■ `indexRotationPeriod` creates time-based index aliases (orders-2024-11, orders-2024-12). This enables efficient ILM (Index Lifecycle Management): automatically move old monthly indexes to warm storage (UltraWarm) after 30 days, delete after 365 days. Without rotation, a single orders index grows indefinitely, degrading OpenSearch performance. Firehose writes to the alias (orders), which points to the current month index.

Complete ShopFlow real-time analytics architecture: events flow from ECS microservices through Kinesis Data Streams to three parallel consumers: Lambda (real-time order processing), Firehose (S3 data lake + OpenSearch), and Flink (fraud detection + trending products). Athena queries S3 for business analytics. Glue crawlers update schemas nightly.

ShopFlow Complete Analytics Architecture

ECS Microservices: order-svc, product-svc, inventory-svc

order-svc: PutRecord to kinesis:shopflow-order-events (8 shards)

product-svc: PutRecord to kinesis:shopflow-clickstream (4 shards)

inventory-svc: PutRecord to kinesis:shopflow-inventory (2 shards)

Kinesis Data Streams: three parallel consumers via EFO

Consumer 1: Lambda ESM (batchSize=100) — order-processor-fn

Process: update order status in Aurora; send SNS notification

DLQ: shopflow-order-dlq (bisectBatchOnError, retryAttempts=3)

Consumer 2: Kinesis Firehose (EFO, dedicated 2MB/s per shard)

S3 Bronze: raw JSON → s3://datalake/raw/orders/YYYY/MM/DD/

S3 Silver: Parquet via Glue nightly ETL job

OpenSearch: bulk index orders (60s buffer, indexRotation=OneMonth)

Consumer 3: Kinesis Analytics Flink (EFO, dedicated 2MB/s per shard)

Fraud Detection: high-value new accounts, repeated failed payments

Trending Products: 5-min tumbling window, >100 views

Inventory Alerts: item count < 10 during flash sale

S3 Data Lake: Bronze / Silver / Gold layered architecture

Glue Crawlers: nightly schema discovery and partition update

Athena: ad-hoc SQL analytics over S3 Parquet (Workgroups per team)

QuickSight: BI dashboards for product, finance, operations teams

Flow Stage	Latency	Reliability	Cost/Month
ECS publish to Kinesis (PutRecord)	~100ms p99	Retry 3x with exponential backoff in SDK	Included in Kinesis shard cost
Kinesis to Lambda (ESM standard polling)	200ms (polling interval)	bisectBatchOnError + DLQ; SLA: process within 5min	Included in Lambda invocation cost
Kinesis to Firehose (EFO push)	60s buffering + S3 PUT latency	ESM is highly durable (11 nines); error backup to DLQ	No Firehose data processing cost
Firehose to OpenSearch (60s buffer)	70s end-to-end (EFO push)	5min 60s buffer; OpenSearch rejection; failed docs to DLQ	Included in Firehose cost

Flow Stage	Latency	Reliability	Cost/Month
Kinesis to Flink (EFO + Flink processing)	25-50ms Flink window size	Exactly-once, idempotent, points every 60s; auto-recover from failures	\$468/mo Kinesis Analytics KPU-hour

18-BP

Best Practices and Anti-Patterns

- ✓ Use userId as partition key for user-scoped events — guarantees in-order processing per user; all events for one user go to same shard
- ✓ Use Enhanced Fan-Out for high-throughput consumers (Firehose, Flink) — dedicated 2MB/s per shard prevents consumer starvation
- ✓ Store events in Parquet format in S3 — 10x smaller than JSON; Athena scans proportionally less data (pay per scan)
- ✓ Partition S3 data lake by year/month/day — Athena prunes partitions; a daily query scans 1/365 of annual data
- ✓ Use On-Demand Kinesis mode for variable traffic (e-commerce spikes) — eliminates ProvisionedThroughputExceededException during Black Friday
- ✓ Set bisectBatchOnError: true on Lambda Kinesis ESM — isolates poison pill records; retries only the half-batch containing the bad record
- ✓ Use Glue Data Catalog for all Athena schemas — single schema registry; crawlers auto-update when S3 schema evolves
- ✓ Enable Kinesis server-side encryption with KMS — customer data in events (user IDs, order amounts) requires encryption at rest
- ✓ Pre-scale Kinesis shards 48 hours before Black Friday for Provisioned mode — shard scaling takes 15-30 minutes; do not wait until traffic spike
- ✓ Use Bronze/Silver/Gold data lake layers — clear data quality boundaries; Silver layer is the reliable source for business analytics
- Never use GetRecords polling when Enhanced Fan-Out is available for latency-sensitive consumers — polling adds 200-2000ms latency vs 70ms EFO push
- Do not use constant partition key (all events to one shard) unless global ordering is required — single shard = 1MB/s limit; uneven distribution
- Avoid small files in S3 data lake — Firehose default 64MB buffer; do not reduce below 5MB; small files degrade Athena scan performance (high overhead per file)
- Do not query Bronze layer (raw JSON) for production reports — schema drift in raw data causes query failures; use Silver (Parquet, validated schema)

18-QR

Quick Reference and Interview Q&A

Concept	Key Value / Rule
Kinesis vs SQS vs SNS	Kinesis: ordered stream; replay; multiple consumers; analytics. SQS: queue; one consumer per message; no replay
Kinesis shard capacity	Write: 1MB/s or 1000 records/s per shard (whichever is hit first). Read: 2MB/s per shard (shared across stream)
Athena cost optimization	\$5/TB scanned. Parquet reduces scan volume 10x vs JSON. Partitioning reduces scan volume by time range
Kinesis record retention	Provisioned: 24 hours default; extends to 7 days (\$0.023/shard-hour for extended). On-Demand: 24 hours
Flink watermarks	Watermarks tell Flink how late data can be before closing a window. Example: INTERVAL 5 SECONDS with LATE DATA
Glue Crawler vs manual schema	Crawler: discovers schema automatically from S3 data; runs on schedule; updates partitions daily. Manual schema